

NUMPY and PANDAS Review

NUMPY and PANDAS Review

What we cover

- What is NumPy?
- Why NumPy?
- Getting started with NumPy
- Indexing and Slicing
- Some Useful NumPy Methods
- What is pandas?
- DataFrame
- pandas Read and Write
- Some Useful pandas Methods



What is NumPy?



- **NumPy** is indeed the fundamental package for scientific computing in Python.
- It provides powerful tools for working with arrays, mathematical functions, linear algebra, random numbers, and more.
- **Anaconda**, a popular Python distribution for data science and scientific computing, does come with NumPy preinstalled by default. This makes it convenient for users to start using NumPy without any additional installation steps.
- NumPy is known for its performance and memory efficiency. It is implemented in C and optimized for numerical operations, making it much faster than Python's built-in data structures for scientific computations. It also provides mechanisms for handling large amounts of data efficiently.

What is NumPy?

- NumPy arrays support indexing and slicing operations, similar to Python lists, strings, and other sequences. This allows you to access specific elements or subarrays within an array, making it easy to manipulate and extract data as needed.
- The core data structure in NumPy is the ndarray (n-dimensional array). It is a homogeneous collection of elements, all of the same data type, which distinguishes it from Python lists that can contain elements of different types. The ndarray provides additional functionality and operations tailored specifically for array computations.
- Overall, NumPy offers a powerful and efficient way to work with numerical data in Python, making it a fundamental tool in scientific computing and data analysis.

NUMPY and PANDAS Review

Why NumPy?

- In Python we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
- Arrays are very frequently used in data science, where speed and resources are very important.

NUMPY and PANDAS Review

NumPy Getting Started

- **Installation:** If you already have **Python** and **PIP** installed on your system, installing NumPy is a simple process. You can install it by executing the following command:

```
C:\Users\Your Name>pip install numpy
```

- Some of Python distribution that already has **NumPy installed** such as **Anaconda** and **Spyder**
- To utilize NumPy in your applications after installing it, simply include the import keyword to import the library: `import numpy`
- At this point, NumPy has been successfully imported and is now available for use in your application.

```
import numpy
```

```
arr = numpy.array([1, 2, 3, 4, 5])
```

```
print(arr, type(arr))
```

```
[1 2 3 4 5] <class 'numpy.ndarray'>
```

NUMPY and PANDAS Review

NumPy Getting Started

- NumPy as np: Typically, NumPy is imported using the np alias.

```
import numpy as np
```

- This allows you to access NumPy's functionality using the np prefix.

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
[1 2 3 4 5]
```

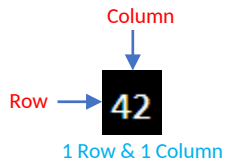
NUMPY and PANDAS Review

Dimensions in Arrays

- In the realm of arrays, a **dimension** denotes a single level of array depth.
- A **nested array** refers to an array that contains arrays as its elements.

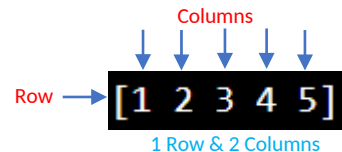
0-D

```
import numpy as np
arr = np.array(42)
print(arr)
```



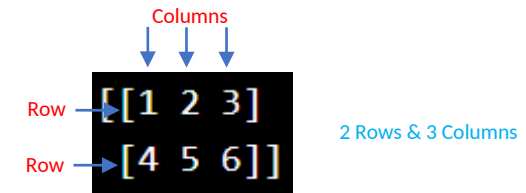
1-D

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```



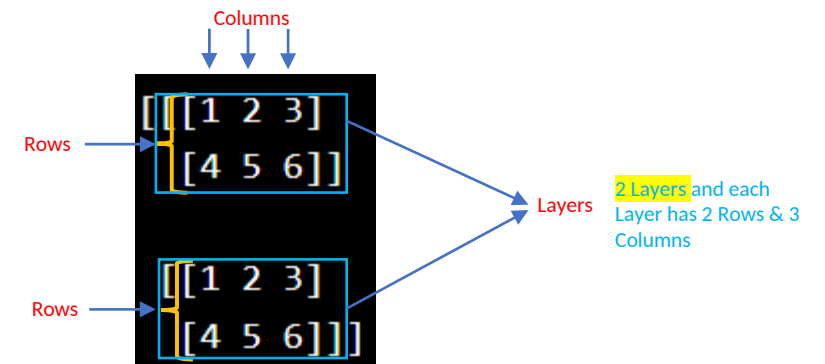
2-D

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
```



3-D

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(arr)
```



NUMPY and PANDAS Review

Dimensions in Arrays

- Checking Dimensions in Arrays: The `ndim` attribute of NumPy arrays returns an integer value indicating the number of dimensions present in the array.

```
import numpy as np

a = np.array(82)
b = np.array([10, 20, 30, 40, 50])
c = np.array([[10, 20, 30], [40, 50, 60]])
d = np.array([[[10, 20, 30], [10, 20, 30]], [[10, 20, 30], [10, 20, 30]]])

print("a-dimension: ",a.ndim)
print("b-dimension: ",b.ndim)
print("c-dimension: ",c.ndim)
print("d-dimension: ",d.ndim)
```

```
a-dimension: 0
b-dimension: 1
c-dimension: 2
d-dimension: 3
```

NUMPY and PANDAS Review

How to Access Array Elements - Indexing

- Array indexing in NumPy is equivalent to accessing an individual element within an array.
- This is achieved by referencing the element's index number.
- In NumPy arrays, the indexing starts from 0, which means that the first element has an index of 0, the second element has an index of 1, and so on.

```
import numpy as np
```

```
arr = np.array([11, 22, 33, 44])
```

```
print('First element in the array is:' ,arr[0])  
print('Second element in the array is:' ,arr[1])  
print('Third element in the array is:' ,arr[2])  
print('Last element in the array is:' ,arr[3])  
print('Last element in the array is:' ,arr[-1])
```

↑
Negative
Indexing

```
First element in the array is: 11  
Second element in the array is: 22  
Third element in the array is: 33  
Last element in the array is: 44  
Last element in the array is: 44
```

NUMPY and PANDAS Review

How to Access Array Elements - Indexing

- Examples:

```
import numpy as np
arr = np.array([3, 2, 5, 7])
print(arr[1] + arr[3])
```

9

```
import numpy as np
arr = np.array([[1,2,3,4,5], [11,12,13,14,15]])
print('2nd element on 1st row: ', arr[0, 1])
print('Last element on 2nd row: ', arr[1, 4])
```

```
2nd element on 1st row:  2
Last element on 2nd row: 15
```

```
import numpy as np
arr = np.array([[[10, 20, 30], [40, 50, 60]], [[70, 80, 90], [100, 110, 120]]])
print(arr[0, 1, 2])
```

60

How to Access Array Elements - Slicing

- **Slicing** in Python refers to extracting elements from an array by specifying a starting and ending index.
- Instead of using individual indices, we use a slice notation: **[start:end]**.
- Additionally, we can include a step value: **[start:end:step]**.
- If no start value is provided, the default is 0.
- If no end value is provided, the default is the length of the array in that dimension.
- If no step value is provided, the default is 1.

NUMPY and PANDAS Review

How to Access Array Elements - Slicing

- Examples:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[1:4])
```

```
[20 30 40]
```

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[:3])
```

```
[10 20 30]
```

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[5:])
```

```
[60 70]
```

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[-4:-2])
```

```
[40 50]
```

NUMPY and PANDAS Review

How to Access Array Elements - Slicing

- Examples:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[1:6:2])
```

```
[20 40 60]
```

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(arr[::2])
```

```
[10 30 50 70]
```

```
import numpy as np
arr = np.array([[10, 20, 30, 40, 50], [60, 70, 80, 90, 100]])
print(arr[1, 0:3])
```

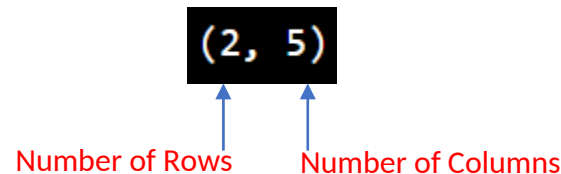
```
[60 70 80]
```

NUMPY and PANDAS Review

How to Get Array Shape

- The **shape of an array** represents the count of elements present in each dimension.
- NumPy arrays possess a **shape attribute**, which returns a tuple containing the count of elements for each respective index.

```
import numpy as np
arr = np.array([[10, 20, 30, 40, 50], [50, 60, 70, 80, 90]])
print(arr.shape)
```



Some Useful NumPy Methods

- The `sort()` function in the NumPy ndarray object allows you to sort a designated array in ascending order.

```
import numpy as np
arr = np.array([30, 25, 4, 10])
print(np.sort(arr))
```

```
[ 4 10 25 30]
```

- To find the indexes in an array that match a specific value, you can utilize the `where()` method. It allows you to search an array and retrieve the indexes where the desired value is found

```
import numpy as np
arr = np.array([15, 4, 3, 40, 4, 4, 6])
x = np.where(arr == 4)
print(x)
```

```
(array([1, 4, 5]),)
```

NUMPY and PANDAS Review

Some Useful NumPy Methods

- In NumPy, the `np.max()` function is used to find the maximum value within an array. It returns the maximum value present in the array, regardless of its dimension.
- The `np.max()` function can also accept additional arguments like `axis` to specify the axis along which the maximum value should be computed.

```
import numpy as np

# Create a NumPy array
arr = np.array([5, 2, 8, 1, 6])

# Find the maximum value in the array
max_value = np.max(arr)

print(max_value) # Output: 8
```

```
import numpy as np

# Create a 2D NumPy array
arr = np.array([[5, 2, 8],
                [1, 6, 4]])

# Find the maximum value in the array
max_value = np.max(arr)

print(max_value) # Output: 8
```

```
import numpy as np

# Create a 2D NumPy array
arr = np.array([[5, 2, 8],
                [1, 6, 4]])

# Find the maximum value along the columns (axis=0)
max_col = np.max(arr, axis=0)

# Find the maximum value along the rows (axis=1)
max_row = np.max(arr, axis=1)

print(max_col) # Output: [5 6 8]
print(max_row) # Output: [8 6]
```

NUMPY and PANDAS Review

Some Useful NumPy Methods

- The `np.min()` function in NumPy is used to find the minimum value within an array. Similar to `np.max()`, it returns the minimum value present in the array, regardless of its dimension.

```
import numpy as np

# Create a 2D NumPy array
arr = np.array([[5, 2, 8],
                [1, 6, 4]])

# Find the minimum value in the array
min_value = np.min(arr)

print(min_value) # Output: 1
```

```
import numpy as np

# Create a 2D NumPy array
arr = np.array([[5, 2, 8],
                [1, 6, 4]])

# Find the minimum value along the columns (axis=0)
min_col = np.min(arr, axis=0)

# Find the minimum value along the rows (axis=1)
min_row = np.min(arr, axis=1)

print(min_col) # Output: [1 2 4]
print(min_row) # Output: [2 1]
```

Some Useful NumPy Methods

- In NumPy, the `np.mean()` function is used to calculate the mean or average value of an array. The mean is computed by summing up all the values in the array and dividing by the total number of elements.

```
import numpy as np

# Create a NumPy array
arr = np.array([5, 2, 8, 1, 6])

# Calculate the mean of the array
mean_value = np.mean(arr)

print(mean_value) # Output: 4.4
```

```
import numpy as np

# Create a 2D NumPy array
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

# Calculate the mean along the columns (axis=0)
mean_col = np.mean(arr, axis=0)

# Calculate the mean along the rows (axis=1)
mean_row = np.mean(arr, axis=1)

print(mean_col) # Output: [2.5 3.5 4.5]
print(mean_row) # Output: [2. 5.]
```

NUMPY and PANDAS Review

Some Useful NumPy Methods

- In NumPy, the `np.std()` function is used to calculate the standard deviation of an array.
- The standard deviation is a measure of the amount of variation or dispersion in a set of values. It provides an indication of how spread out the values in the array are from the mean (average).

```
import numpy as np

# Create a NumPy array
arr = np.array([5, 2, 8, 1, 6])

# Calculate the standard deviation of the array
std_value = np.std(arr)

print(std_value) # Output: 2.588...
```

NUMPY and PANDAS Review

Some Useful NumPy Methods

- In NumPy, the `np.shape()` function is used to determine the shape or dimensions of an array or matrix.
- It returns a tuple containing the lengths of each dimension of the array.

```
import numpy as np

# Create a 2D array
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Get the shape of the array
shape = np.shape(arr)

# Print the shape
print(shape)
```

Output:

```
(2, 3)
```

Some Useful NumPy Methods

- The `np.reshape()` function in NumPy is used to change the shape of an array without altering its data.
- It allows you to specify the new dimensions of the array, as long as the total number of elements remains the same.
- Here's the general syntax for using `np.reshape()`: `np.reshape(array, new_shape)`
- The `array` parameter represents the original array that you want to reshape, and the `new_shape` parameter specifies the desired new shape of the array. The `new_shape` can be provided as a tuple or as separate integer arguments.

NUMPY and PANDAS Review

Some Useful NumPy Methods

- The `np.reshape()` example:

```
import numpy as np

# Create a 1D array
arr = np.array([1, 2, 3, 4, 5, 6])

# Reshape the array to a 2D matrix with 2 rows and 3 columns
reshaped_arr = np.reshape(arr, (2, 3))

# Print the reshaped array
print(reshaped_arr)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

NUMPY and PANDAS Review

Some Useful NumPy Methods

- In NumPy, the `flatten()` function is used to create a copy of an array and convert it into a 1-dimensional (flat) array.
- It returns a new array that contains all the elements of the original array in a continuous sequence.
- `reshape` and `flatten` when working with multi-dimensional data (such as images)

```
import numpy as np

# Create a 2D array
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Flatten the array
flattened_arr = arr.flatten()

# Print the flattened array
print(flattened_arr)
```

Output:

```
[1 2 3 4 5 6]
```

Some Useful NumPy Methods

- In NumPy, `np.vstack()` and `np.hstack()` are functions used to vertically stack and horizontally stack arrays, respectively.
- `np.vstack()`: This function is used to vertically stack arrays. It takes a sequence of arrays as input and stacks them vertically to form a new array. The arrays must have the same number of columns or dimensions along the axis being stacked. The resulting array has the same number of columns as the input arrays and the total number of rows is the sum of the rows in each input array.
- `np.hstack()`: This function is used to horizontally stack arrays. It takes a sequence of arrays as input and stacks them horizontally to form a new array. The arrays must have the same number of rows or dimensions along the axis being stacked. The resulting array has the same number of rows as the input arrays and the total number of columns is the sum of the columns in each input array.

NUMPY and PANDAS Review

Some Useful NumPy Methods

- `np.vstack()` and `np.hstack()` examples:

```
import numpy as np

# Create two arrays
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# Vertically stack the arrays
vstacked_arr = np.vstack((arr1, arr2))

# Horizontally stack the arrays
hstacked_arr = np.hstack((arr1, arr2))

# Print the stacked arrays
print("Vertically stacked array:")
print(vstacked_arr)
print("\nHorizontally stacked array:")
print(hstacked_arr)
```

Output:

```
Vertically stacked array:
[[1 2 3]
 [4 5 6]]

Horizontally stacked array:
[1 2 3 4 5 6]
```

What is pandas?



- **pandas** is a fast, powerful, flexible and easy to use open-source data analysis and manipulation tool, built on top of the Python programming language.
- It is widely used for working with structured data.
- Pandas introduces two primary data structures, namely **Series** and **DataFrame**, which offer efficient ways to handle and manipulate data.
- **Series:** A Series is a one-dimensional labeled array that can hold any data type. It is similar to a column in a spreadsheet or a database table. Each element in a Series has a unique label called an index.
- **DataFrame:** A DataFrame is a two-dimensional labeled data structure that represents a tabular, spreadsheet-like format. It consists of rows and columns, where each column can be of a different data type. It is analogous to a table in a relational database or a spreadsheet.

What is pandas?



- A DataFrame Pandas provides a wide range of functionalities, including:
 - **Data cleaning and preprocessing:** Pandas offers various methods for handling missing data, removing duplicates, transforming data types, and performing other data cleaning operations.
 - **Data manipulation:** It provides flexible tools for selecting, filtering, sorting, and transforming data within a DataFrame. You can perform operations like merging, joining, and reshaping data.
 - **Data analysis and statistics:** Pandas allows you to perform statistical computations, descriptive statistics, aggregations, and apply mathematical operations on data. It also supports time series analysis and handling of categorical variables.
 - **Data input/output:** Pandas supports reading and writing data in various formats, such as CSV, Excel, SQL databases, JSON, and more.
- Pandas is widely used in data analysis, data science, and machine learning workflows. It provides an intuitive and efficient interface for handling and manipulating structured data, making it an essential tool in the Python ecosystem for data professionals and researchers.

NUMPY and PANDAS Review

DataFrame



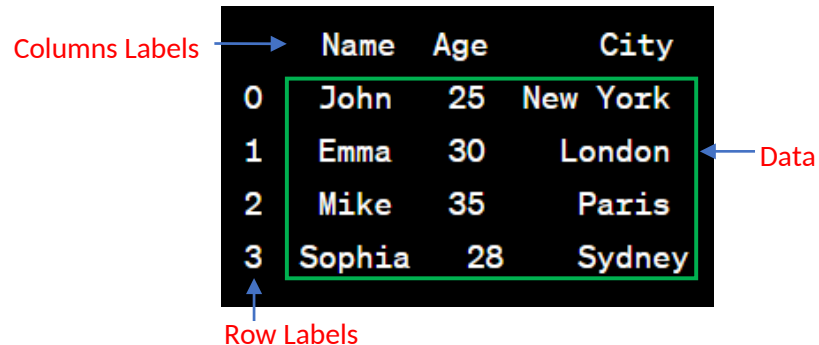
- Example of creating a Python DataFrame using pandas:

```
import pandas as pd

# Create a dictionary of data
data = {
    'Name': ['John', 'Emma', 'Mike', 'Sophia'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'London', 'Paris', 'Sydney']
}

# Create a DataFrame from the dictionary
df = pd.DataFrame(data)

# Print the DataFrame
print(df)
```



A diagram illustrating the structure of a pandas DataFrame. It shows a table with four rows and three columns. The columns are labeled 'Name', 'Age', and 'City'. The rows are indexed from 0 to 3. A green rectangular box highlights the data cells, excluding the row and column labels. Annotations with arrows point to these components: 'Columns Labels' points to the column headers, 'Row Labels' points to the row indices, and 'Data' points to the green-bordered data area.

	Name	Age	City
0	John	25	New York
1	Emma	30	London
2	Mike	35	Paris
3	Sophia	28	Sydney

NUMPY and PANDAS Review

DataFrame

- To access data in a DataFrame in pandas, you can use various indexing and slicing techniques.
- Columns in a DataFrame are typically accessed using **keys**, which represent the column or row labels.

```
import pandas as pd

# Create a DataFrame
data = {
    'Name': ['John', 'Emma', 'Mike', 'Sophia'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'London', 'Paris', 'Sydney']
}
df = pd.DataFrame(data)

# Access the 'Name' column
names = df['Name']
print("Accessing a Column:")
print(names)
print()

# Access multiple columns
columns = df[['Name', 'Age']]
print("Accessing Multiple Columns:")
print(columns)
print()

# Access a specific row by index
row = df.loc[2]
print("Accessing a Specific Row:")
print(row)
print()

# Access a subset of data using slicing
subset = df.loc[1:3, 'Age':'City']
print("Accessing a Subset of Data:")
print(subset)
```

```
Accessing a Column:
0    John
1    Emma
2    Mike
3    Sophia
Name: Name, dtype: object
```

```
Accessing Multiple Columns:
   Name  Age
0  John   25
1  Emma   30
2  Mike   35
3  Sophia  28
```

```
Accessing a Specific Row:
Name    Mike
Age      35
City    Paris
Name: 2, dtype: object
```

```
Accessing a Subset of Data:
   Age  City
1   30 London
2   35  Paris
3   28 Sydney
```

DataFrame: Read



- **pandas** provides various read methods that allow you to read data from different file formats and sources. Here are some commonly used read methods in Pandas:
 - **pd.read_csv():** Reads data from a CSV (Comma-Separated Values) file into a DataFrame.
 - **pd.read_excel():** Reads data from an Excel file into a DataFrame.
 - **pd.read_json():** Reads data from a JSON (JavaScript Object Notation) file into a DataFrame.
 - **pd.read_sql():** Reads data from a SQL database query into a DataFrame.
 - **pd.read_html():** Reads data from an HTML file or webpage into a list of DataFrames.
 - **pd.read_clipboard():** Reads data from the clipboard into a DataFrame.
- These methods offer various parameters to customize the reading process, such as specifying file paths, sheet names, column names, delimiter options, and more.

NUMPY and PANDAS Review

DataFrame: Read



- Example:

```
import pandas as pd

# Read data from a CSV file
df = pd.read_csv('data.csv')

# Print the DataFrame
print(df)
```

Output

	Name	Age	City
0	John	25	New York
1	Emma	30	London
2	Mike	35	Paris
3	Sophia	28	Sydney

- This snapshot shows the content of data.csv

```
CSV
Name, Age, City
John, 25, New York
Emma, 30, London
Mike, 35, Paris
Sophia, 28, Sydney
```

NUMPY and PANDAS Review

DataFrame: Read



- The `myData.drop()` method in pandas is used to remove specific rows or columns from a DataFrame. It returns a new DataFrame with the specified rows or columns removed.

```
@author: Mehdi
"""
import pandas as pd

# Create a DataFrame
data = {
    'Name': ['John', 'Emma', 'Mike', 'Sophia'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'London', 'Paris', 'Sydney']
}
df = pd.DataFrame(data)

# Drop a specific row by index
new_df = df.drop(2)
print(new_df)

# Drop a specific column by name
new_df = df.drop('Age', axis=1)
print('\n', new_df)

# Drop multiple rows by index
new_df = df.drop([1, 3])
print('\n', new_df)

# Drop multiple columns by name
new_df = df.drop(['Age', 'City'], axis=1)
# Print the new DataFrame
print('\n', new_df)
```

Output:

	Name	Age	City
0	John	25	New York
1	Emma	30	London
3	Sophia	28	Sydney

	Name	City
0	John	New York
1	Emma	London
2	Mike	Paris
3	Sophia	Sydney

	Name	Age	City
0	John	25	New York
2	Mike	35	Paris

	Name
0	John
1	Emma
2	Mike
3	Sophia

DataFrame: Write



- The Pandas provides several write methods that allow you to save DataFrame data to various file formats. Here are some commonly used write methods in Pandas:
 - `df.to_csv()`: Writes DataFrame to a CSV (Comma-Separated Values) file.
 - `df.to_excel()`: Writes DataFrame to an Excel file.
 - `df.to_json()`: Writes DataFrame to a JSON (JavaScript Object Notation) file.
 - `df.to_sql()`: Writes DataFrame to a SQL database table.
 - `df.to_html()`: Writes DataFrame to an HTML file or string.
 - `df.to_clipboard()`: Writes DataFrame to the clipboard.
- These methods allow you to customize the writing process by specifying file paths, sheet names, column names, index inclusion, and more.

NUMPY and PANDAS Review

DataFrame: Write



- Example:

```
import pandas as pd

# Create a DataFrame
data = {
    'Name': ['John', 'Emma', 'Mike', 'Sophia'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'London', 'Paris', 'Sydney']
}
df = pd.DataFrame(data)

# Write DataFrame to a CSV file
df.to_csv('output.csv', index=False)
```

```
Name, Age, City
John, 25, New York
Emma, 30, London
Mike, 35, Paris
Sophia, 28, Sydney
```

NUMPY and PANDAS Review

DataFrame: Write



- If the data is in a format other than a pandas DataFrame, such as a NumPy array or other types, it needs to be converted into a Pandas DataFrame using the `pandas.DataFrame()` function before using the `to_csv()` method. This ensures that the data is in the appropriate format for writing to a CSV file using Pandas.

```
import pandas as pd
import numpy as np

# Create a NumPy array
my_array = np.array([[1, 2, 3], [4, 5, 6]])

# Convert the NumPy array to a Pandas DataFrame
df = pd.DataFrame(my_array, columns=['A', 'B', 'C'])

# Save the DataFrame to a CSV file
df.to_csv('output.csv', index=False)
```

Some Useful pandas Methods



- The `df.describe()` method in pandas is used to generate descriptive statistics of a DataFrame. It provides a summary of the central tendency, dispersion, and shape of the distribution of numerical columns in the DataFrame.
 - **Count:** The number of non-null values in each column.
 - **Mean:** The arithmetic mean (average) of each column.
 - **Standard Deviation:** The standard deviation, which measures the spread of values around the mean.
 - **Minimum:** The minimum value in each column.
 - **25th Percentile:** The value below which 25% of the data falls.
 - **Median:** The median (50th percentile) of each column.
 - **75th Percentile:** The value below which 75% of the data falls.
 - **Maximum:** The maximum value in each column.
- By default, `df.describe()` only includes the numeric columns of the DataFrame. However, you can include all columns by using the `include='all'` parameter.

NUMPY and PANDAS Review

Some Useful pandas Methods



- The `df.describe()` example:

```
import pandas as pd

# Create a DataFrame
data = {
    'Name': ['John', 'Emma', 'Mike'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Paris']
}
df = pd.DataFrame(data)

# Generate descriptive statistics
statistics = df.describe()

print(statistics)
```

Output:

	Age
count	3.000000
mean	30.000000
std	5.000000
min	25.000000
25%	27.500000
50%	30.000000
75%	32.500000
max	35.000000

Some Useful pandas Methods



- The `df.sort_index()` method in pandas is used to sort the DataFrame based on the row index. It rearranges the rows of the DataFrame in ascending or descending order based on the index values.

```
@author: Mehdi
"""

import pandas as pd

# Create a DataFrame
data = {
    'Name': ['John', 'Emma', 'Mike'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Paris']
}
df = pd.DataFrame(data, index=[2, 0, 1])

print(df)
# Sort the DataFrame based on the row index
sorted_df = df.sort_index()

print('\n',sorted_df)
```

Output:

	Name	Age	City
2	John	25	New York
0	Emma	30	London
1	Mike	35	Paris

	Name	Age	City
0	Emma	30	London
1	Mike	35	Paris
2	John	25	New York

NUMPY and PANDAS Review

Some Useful pandas Methods



- The `df.sort_values()` method is used to sort a DataFrame based on the values in one or more columns. It rearranges the rows of the DataFrame in ascending or descending order based on the specified column values.

```
import pandas as pd

# Create a DataFrame
data = {
    'Name': ['John', 'Emma', 'Mike'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Paris']
}
df = pd.DataFrame(data)

# Sort the DataFrame based on the 'Age' column in ascending order
sorted_df = df.sort_values(by='Age')

print(sorted_df)
```

Output:

	Name	Age	City
0	John	25	New York
1	Emma	30	London
2	Mike	35	Paris

NUMPY and PANDAS Review

What we covered

- What is NumPy? ☒
- Why NumPy? ☒
- Getting started with NumPy ☒
- Indexing and Slicing ☒
- Some Useful NumPy Methods ☒
- What is pandas? ☒
- DataFrame ☒
- pandas Read and Write ☒
- Some Useful pandas Methods ☒

NUMPY and PANDAS Review

References

- www.numpy.org/doc/
- <https://pandas.pydata.org/docs/>